
Lossless Source Coding

Luping, Xiang (向路平)

luping.xiang@nju.edu.cn

Office: 南雍楼西231

English Language

Char	Prob	Char	Prob	Char	Prob	Char	Prob
␣	0.2	I	0.055	C	0.023	B	0.0105
E	0.105	R	0.054	F,U	0.0225	V	0.008
T	0.072	S	0.052	M	0.021	K	0.003
O	0.0654	H	0.047	P	0.0175	X	0.002
A	0.063	D	0.035	Y,W	0.012	J,Q	0.001
N	0.059	L	0.029	G	0.011	Z	0.001

□ Modelling English with Markov info sources having different orders, we may obtain the approximate entropy of English:

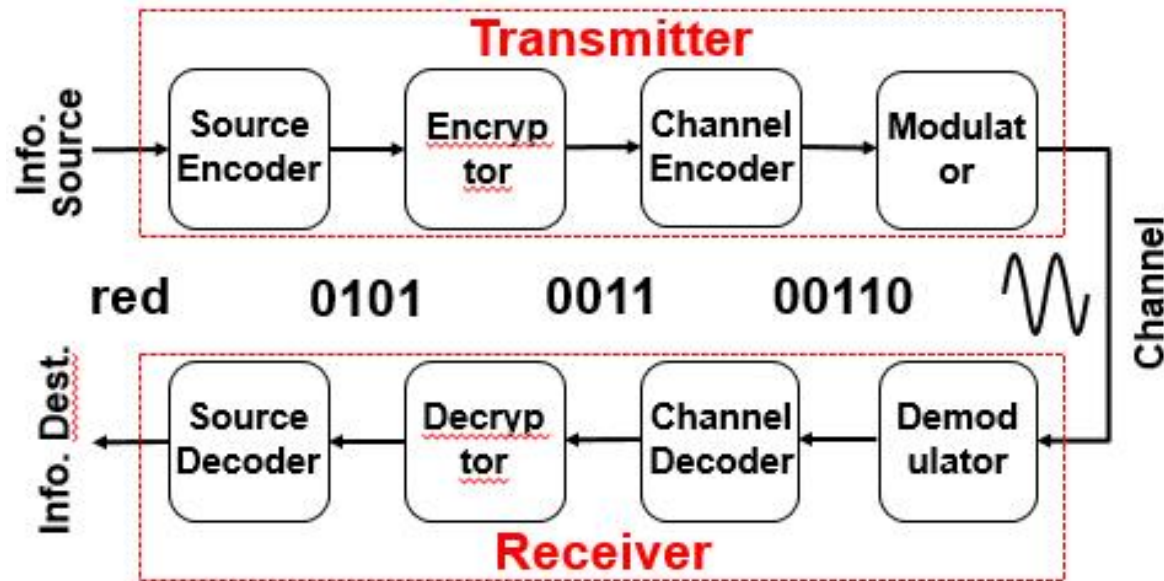
$$H_0 = 4.75 \text{ bit/char} \quad H_1 = 4.03 \text{ bit/char}$$

$$H_2 = 3.32 \text{ bit/char} \quad \dots \quad H_\infty = 1.4 \text{ bit/char}$$

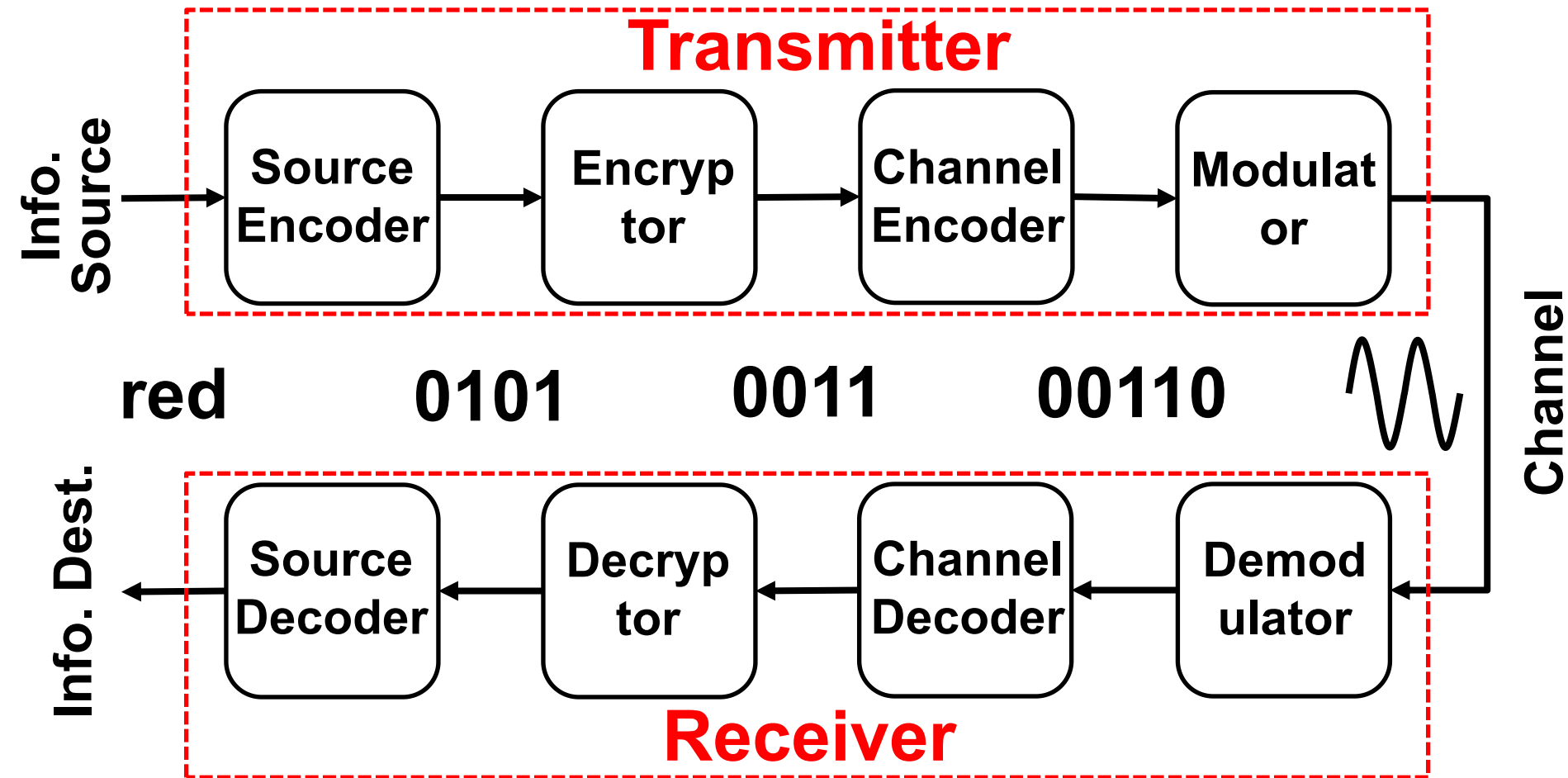
What is the communication process if we want to transmit the coin/dice toss results?



Source Coding: Bitmap



Source Coding: Vector Graph





Bitmap

VS



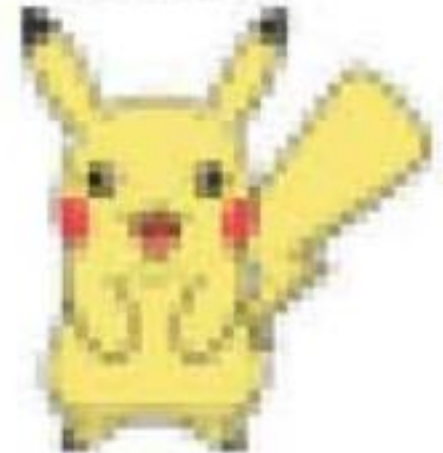
Vector

www.educba.com

Vector



Bitmap



Data Compression in Video Chatting



- ❑ In the video chatting, we do not need to transmit every details of the bodies and background in a frame. We only need to transmit the image of the head. Furthermore, we only need to transmit the offset vector rather than the details of all the pixels.

Morse Code Alphabet

A	.-	B	-...	C	-.-.
D	-..	E	.	F	..-.
G	--.	H		I	..
J	.---	K	-.-	L	.-..
M	--	N	-.	O	---
P	.-.	Q	--.-	R	.-.
S	...	T	-	U	..-
V	...-	W	.--	X	-.-.
Y	-.--	Z	--..		

0	-----	1	.----	2	..---
3	...--	4	-	5	
6	-....	7	--...	8	---..
9	-----.				

.	.-.-.-	,	--.-.-	?	..-.-.
'	.-----	!	-.--.-	/	-.--.
(	-.--.	)	-.--.-	&	.-...
:	---...	;	-.--.-	=	-.--.
+	.-.-.-	-	-.---.	_	..-.-.
"	.-.-.-	\$	...-.-.	@	.-.-.-
¿	..-.-.	¡	--...-		

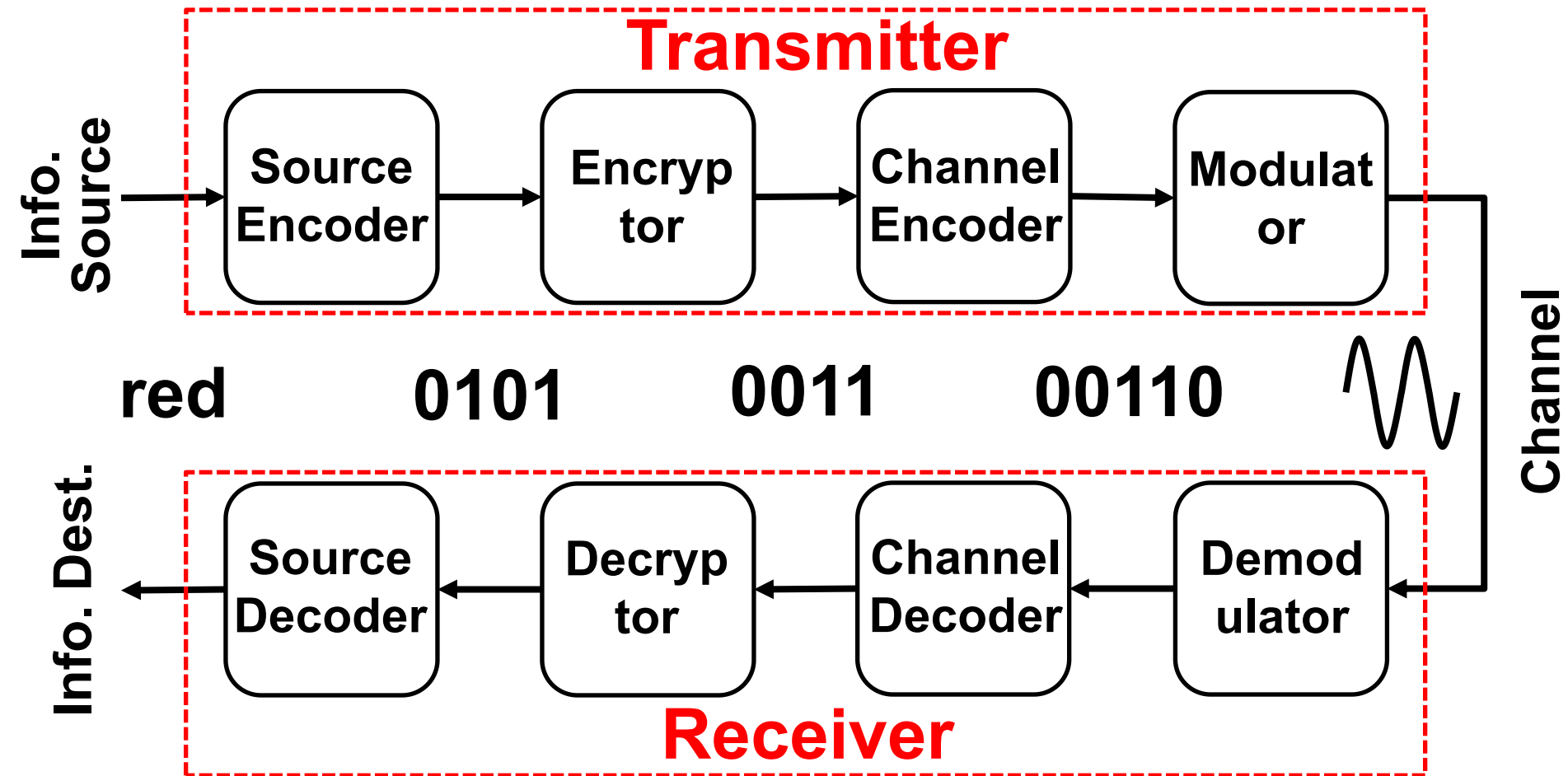
Main Content

1 Fundamental of Source Coding

2 Lossless Source Coding Theorems

3 Classic Source Coding Schemes

Communication System



Fundamental Concepts

- ❑ **Source coding:** reduce the redundancy, transmit more information with a lower rate, improve the transmission **efficiency**.
- ❑ **Channel coding:** increase the redundancy by increasing the code rate or the bandwidth, improve the transmission **reliability**.
- ❑ **Cryptogram:** encryption and decryption aims for increasing and reducing the entropy, respectively, improve the transmission **security**.
- ❑ **Source coding theorem:** lossless source coding theorem (discrete info source) and lossy source coding (continuous info source).
- ❑ **Lossless source coding:** every symbol generated by the info source should be mapped onto a specific codeword output the encoder.
- ❑ **Lossy source coding:** by satisfying the communication requirement, the source coding allows some distortion (rate distortion theory).

Definition of Source Coding

- ❑ **Source Coding:** representing the original messages of the info source by the **code sequences**, which are more suitable to be transmitted on a specific medium. The source coding is completed by the **encoder**.
- ❑ The random message $\mathbf{X}$ generated by the info source consists of L discrete symbols, which is expressed as

$$\mathbf{X} = (X_1 X_2 \cdots X_l \cdots X_L), \quad \text{where we have } X_l = \{a_1, a_2, \cdots, a_i, \cdots a_n\}.$$

Source coding transforms the original random symbol sequence $\mathbf{X}$ into the following code sequence $\mathbf{Y}$:

$$\mathbf{Y} = (Y_1 Y_2 \cdots Y_k \cdots Y_K), \quad \text{where we have } Y_k = \{b_1, b_2, \cdots, b_j, \cdots b_m\}.$$

Lossless coding: every message of the info source is encoded into a specific code sequence (**codeword**); Every codeword can only be decoded into a specific message. **Loss Entropy $H(\mathbf{X}|\mathbf{Y})$**



English Language

Char	Prob	Char	Prob	Char	Prob	Char	Prob
␣	0.2	I	0.055	C	0.023	B	0.0105
E	0.105	R	0.054	F,U	0.0225	V	0.008
T	0.072	S	0.052	M	0.021	K	0.003
O	0.0654	H	0.047	P	0.0175	X	0.002
A	0.063	D	0.035	Y,W	0.012	J,Q	0.001
N	0.059	L	0.029	G	0.011	Z	0.001

□ Modelling English with Markov info sources having different orders, we may obtain the approximate entropy of English:

$$H_0 = 4.75 \text{ bit/char} \quad H_1 = 4.03 \text{ bit/char}$$

$$H_2 = 3.32 \text{ bit/char} \quad \dots \quad H_\infty = 1.4 \text{ bit/char}$$

Coding Examples

Message of Info source	Sending Probability	Code A	Code B	Code C	Code D
<i>B</i>	0.0105	0	0	0	0
<i>E</i>	0.105	0	1	01	10
<i>X</i>	0.002	1	00	001	110
<i>Z</i>	0.001	10	11	111	1110

- ❑ What are the pros and cons of different codes?
- ❑ When the encoded bit sequence is 0010...

Source Coding Theorem

- ❑ **Message:** any abstract symbols, such as letters and symbols and etc.
- ❑ **Code Element:** symbol that can be identified by communication system, such as the binary symbol 0/1.
- ❑ **Key problem:** how many code elements are used for encoding the messages of the information source for the error-free decoding?
- ❑ The codeword length is related to the information rate.
- ❑ **Source coding theorem** aims for **finding the minimum information rate for realizing the error-free decoding.**

Lossless Source Coding {

- Fixed length Code:** The codeword length is fixed. Fixed length coding theorem aims for finding the minimum fixed length K .
- Varied Length Code:** The codeword length is varied. Varied length coding theorem aims for finding the minimum expected codeword length $\bar{K}$.

Coding Efficiency and Error Rate

- We have a stationary memoryless info source $\mathbf{X}$, which sequentially sends symbols by obeying the following probability distribution:

$$\left(\begin{array}{c} \mathbf{X} \\ p(\mathbf{X}) \end{array} \right) = \left\{ \begin{array}{cccc} a_1 & a_2 & a_3 & a_4 \\ 1/2 & 1/4 & 1/8 & 1/8 \end{array} \right\}$$

Please calculate the coding efficiency and the decoding error rate for the following cases:

- (1) Encoding the one-symbol messages sent by $\mathbf{X}$ with a binary code having fixed length 2.
- (2) Encoding the two-symbol messages sent by $\mathbf{X}$ with a binary code having fixed length 3.

Unique Decodability

- Definition:
 - For every string of source letters $\{x_1, x_2, \dots, x_n\}$, the encoded output $\{C(x_1)C(x_2), \dots, C(x_n)\}$ must be distinct, i.e., must differ from $\{C(x'_1)C(x'_2), \dots, C(x'_m)\}$ for any other source string $\{x'_1, x'_2, \dots, x'_m\}$.
- Decoding error:
 - If $C(x_1)C(x_2), \dots, C(x_n) = C(x'_1)C(x'_2), \dots, C(x'_m)$, decoder must fail on one of these inputs.
- Example:
 - Consider $a \rightarrow 0, b \rightarrow 01, c \rightarrow 10$
 - Then $ac \rightarrow 010$ and $ba \rightarrow 010$
 - Not uniquely decodable.

Example of Source coding

- **Fixed length coding versus variable length coding**

Source symbol	Prob. of source symbol	Codebook for source coding				
		Code 1	Code 2	Code 3	Code 4	Code 5
s_1	$p(s_1)=1/2$	00	00	0	1	1
s_2	$p(s_2)=1/4$	01	11	01	10	01
s_3	$p(s_3)=1/8$	10	00	001	100	001
s_4	$p(s_4)=1/8$	11	11	0001	1000	0001

- **Is each code sequence decodable uniquely?**
- **Which has highest efficiency?**
 - **the average number of coded bits per source symbol**

Fixed Length Coding Theorem

- The entropy rate of a stationary memoryless info source X is $H(X)$

$$\begin{array}{ccc} X = (X_1 X_2 \cdots X_l \cdots X_L) & \text{Fixed length coding} & Y = (Y_1 Y_2 \cdots Y_k \cdots Y_K) \\ X_l = \{a_1, a_2, \cdots, a_i, \cdots a_n\} & \rightarrow & Y_k = \{b_1, b_2, \cdots, b_j, \cdots b_m\} \end{array}$$

- Fixed length coding theorem: for arbitrary $\varepsilon > 0$, $\delta > 0$, if we have

Information Rate (bit/symbol) $R = \frac{K}{L} \log_2 m \geq H(X) + \varepsilon$ (Positive Theorem)

When L is high, the decoding error could be lower than δ . Otherwise,

Information Rate (bit/symbol) $R = \frac{K}{L} \log_2 m \leq H(X) - 2\varepsilon$ (Negative Theorem)

When L is high, the decoding error cannot be avoided.

Ideal Encoder

- ❑ Source coding theorem (positive): when information rate is a little higher than the entropy, we may realise error-free decoding. The length L of the message generated by the info source has to satisfy:

$$L \geq \frac{\text{Var}[I(a_i)]}{\varepsilon^2 \delta}.$$

The decoding error rate is lower than an arbitrary positive number δ .

- ❑ Fixed length coding theorem (negative): when the information rate R is lower than the single-symbol entropy $H(X)$ by ε , the decoding error rate may not exceed δ . If R is lower than $H(X)$ by 2ε , the decoding error rate must be higher than δ .
- ❑ When we encode the original message having an infinite length ($L \rightarrow \infty$), the ideal encoder having a coding efficiency tending to a unity exist, namely

$$\lim_{L \rightarrow \infty} H(X) / \frac{K}{L} \log_2 m = 1,$$

which is impossible in practice.

Discussion on the Ideal Encoder

□ We have the following single-symbol info source:

$$\begin{pmatrix} X \\ p(X) \end{pmatrix} = \begin{Bmatrix} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 & a_8 \\ 0.4 & 0.18 & 0.1 & 0.1 & 0.07 & 0.06 & 0.05 & 0.04 \end{Bmatrix}.$$

Please encode this info source by using the fixed length code.

- If we use the binary fixed length code to encode every single symbol generated by the info source, find the coding efficiency.
- If we only need to achieve the decoding error rate 10^{-6} , what is the minimum length of the symbol sequences that needs to be encoded together? What is the coding efficiency then?
- Please calculate the length of the symbol sequence that needs to be encoded together, when the coding efficiency is 90%, the decoding error rate is 10^{-6}

How an ideal encoder operates?

❑ An ideal encoder operates by obeying the following steps:

(1) Given ε and δ , calculate the length L of the symbol sequence generated by the info source that need to be encoded together by exploiting $L \geq \frac{\text{Var}[I(a_i)]}{\varepsilon^2 \delta}$.

(2) Given L , calculate the probabilities of all the possible symbol sequences having a length of L .

(3) Sort all the possible symbol sequences having a length of L by the descending order of their sending probabilities.

(4) Encode the sorted symbol sequences. The encoded symbol sequences constitute a set of A_ε . The encoding process continues until $p(A_\varepsilon) \geq 1 - \delta$.

Varied Length Code

- For a memoryless info source X having an entropy of $H(X)$, when encoding its symbols by an m -ary varied-length codeword, a lossless encoding scheme exists, whose average codeword length $\bar{K}$ satisfies:

$$\frac{H(X)}{\log_2 m} \leq \bar{K} < 1 + \frac{H(X)}{\log_2 m}.$$

The average information rate of the encoder satisfies:

$$H(X) \leq \bar{R} = \frac{\bar{K}}{L} \log_2 m < H(X) + \varepsilon,$$

Where we have $L = 1$ and ε is an arbitrary positive.

- When the info source sends a symbol sequence having a length of L , the average code length satisfies:

$$\frac{LH(X)}{\log_2 m} \leq \bar{K} < 1 + \frac{LH(X)}{\log_2 m}$$

Coding Efficiency

- ❑ The length L of the symbol sequence to be encoded together by a varied length encoder is far lower than a fixed length counterpart. The coding efficiency is lower-bounded by

$$\eta = \frac{H(X)}{\bar{R}} > \frac{H(X)}{H(X) + \frac{\log_2 m}{L}}$$

- ❑ Encode the following single-symbol source by using a binary varied length code:

$$\left(\begin{matrix} X \\ p(X) \end{matrix} \right) = \left\{ \begin{matrix} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 & a_8 \\ 0.4 & 0.18 & 0.1 & 0.1 & 0.07 & 0.06 & 0.05 & 0.04 \end{matrix} \right\}.$$

When the coding efficiency is required to be higher than 90%, calculate the length of the symbol sequences that needs to be processed together.

Pros and Cons of Varied-Length Code

- ❑ **Basic principle of varied-length coding:** the messages having high sending probabilities are encoded into short codewords, while the messages having low sending probabilities are encoded into long codewords. Therefore, the average codeword length can be minimised in order to increase the communication efficiency.
- ❑ **Varied-length coding increase the complexity of decoding:** The decoder should correctly identify the start of the codewords having different lengths (**synchronous decoding**); since the code has various length, when a specific code element is received, the decoder does not know whether this is the end of a code. It has to wait for receiving the following code elements. Then it can make a correct decoding (**decoding delay**).

Review

Average codeword length $\bar{K}$ satisfies:

$$\frac{H(X)}{\log_2 m} \leq \bar{K} < 1 + \frac{H(X)}{\log_2 m}.$$

$$H(X) \leq \bar{R} = \frac{\bar{K}}{L} \log_2 m < H(X) + \varepsilon,$$

Review example

□ Encode the following single-symbol source by using

(1) a binary varied length code

(2) a binary fixed length code

$$\left(\begin{matrix} X \\ p(X) \end{matrix} \right) = \left\{ \begin{matrix} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 & a_8 \\ 0.4 & 0.18 & 0.1 & 0.1 & 0.07 & 0.06 & 0.05 & 0.04 \end{matrix} \right\}.$$

When the coding efficiency is required to be higher than 90%, calculate the length of the symbol sequences that needs to be processed together.

$$\eta = \frac{H(X)}{H(X) + \varepsilon}, \quad L \geq \frac{\text{Var}[I(a_i)]}{\varepsilon^2 \delta}.$$

$$\eta = \frac{H(X)}{\bar{R}} > \frac{H(X)}{H(X) + \frac{\log_2 m}{L}}$$

$$H(X) = 2.5525 \text{ bit/sign}$$

Review example

- Please encode the following single symbol info source by
- (1) binary Shannon code;
 - (2) binary Fano code;
 - (3) binary Huffman code.

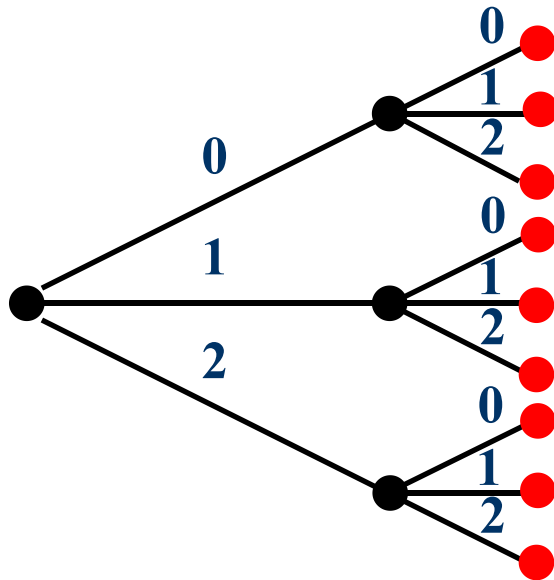
$$\begin{pmatrix} X \\ P(X) \end{pmatrix} = \left\{ \begin{matrix} a_1 & a_2 & a_3 & a_4 \\ 0.25 & 0.4 & 0.2 & 0.15 \end{matrix} \right\}$$

Examples of Varied Length Coding

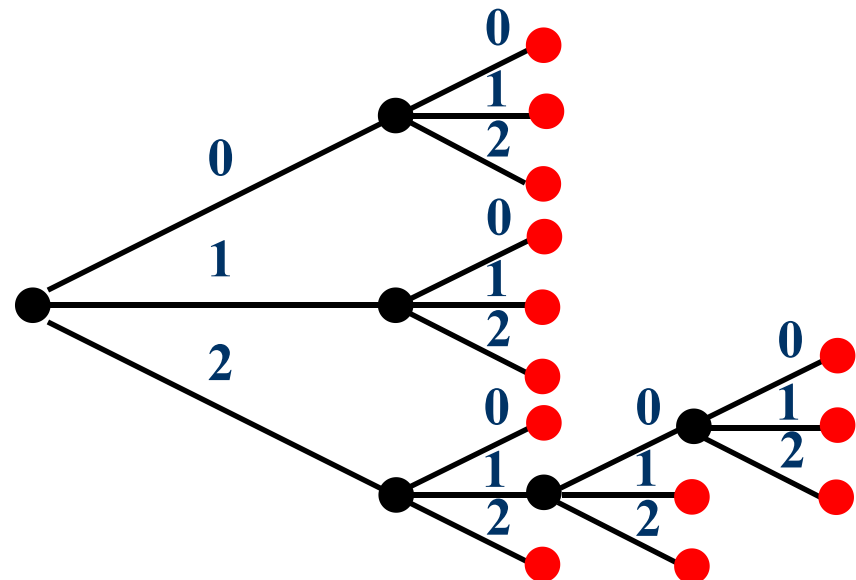
Message of Info source	Sending Probability	Code A	Code B	Code C	Code D
a_1	0.5	0	0	0	0
a_2	0.25	0	1	01	10
a_3	0.125	1	00	011	110
a_4	0.125	10	11	0111	1110
		Undecodable	Undecodable	Decodable Decoding Delay	Decodable No Delay

❑ A code is called a prefix-free code if no codeword is a prefix of any other codeword. For brevity, a prefix-free code will be referred to as a **prefix code**, which is also regarded as an **instantaneous code**.

Tree Graph of Prefix Code



Full Tree



Non-full Tree

- ❑ Prefix codes can be constructed by a tree graph. The starting point is the root and every segment between a pair of nodes is branch.
- ❑ The full tree constructs the **fixed length code**, while the non-full tree constructs the **varied length code**.

Necessary and Sufficient Conditions

□ Necessary and sufficient condition of an m -ary prefix code is

$$\sum_{i=1}^n m^{-k_i} \leq 1. \quad (\text{Kraft Inequality})$$

k_i is the length of the i -th codeword.

Proof: Prove the necessity and the sufficiency of the inequality.

□ Prefix codes are uniquely decodable.

□ Given an info source $\left(\begin{smallmatrix} X \\ p(X) \end{smallmatrix} \right) = \left\{ \begin{smallmatrix} a_1 & \cdots & a_i & \cdots & a_n \\ p(a_1) & \cdots & p(a_i) & \cdots & p(a_n) \end{smallmatrix} \right\}$, if the length k_i of the message a_i 's codeword satisfies

$$1 - \frac{\log_2 p(a_i)}{\log_2 m} > k_i \geq -\frac{\log_2 p(a_i)}{\log_2 m},$$

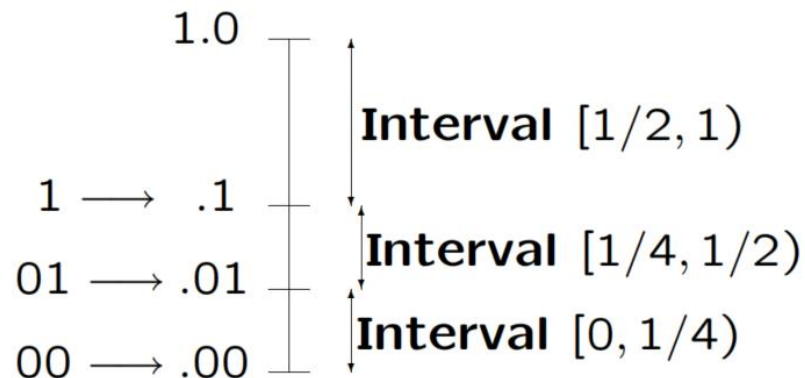
Then this is a prefix code.

Proof on Kraft Inequality

- We prove this by associating codewords with base 2 expansions i.e., ‘decimals’ in base 2.

Represent binary codeword $y_1, y_2, \dots, y_m$ as

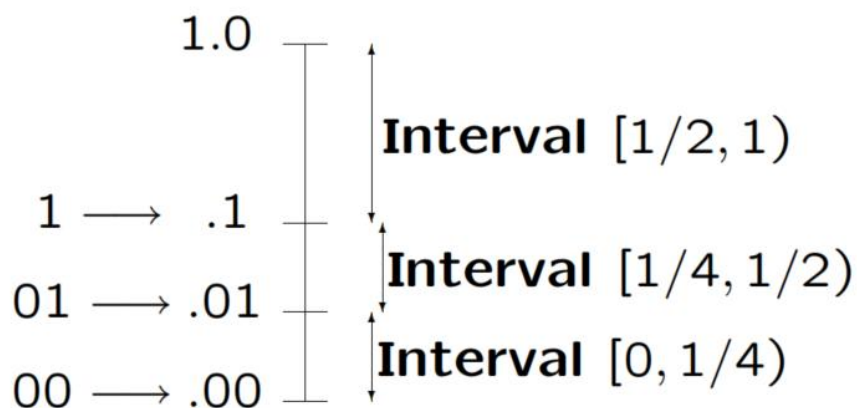
$$.y_1y_2 \cdots y_m = y_1/2 + y_2/4 + \cdots + y_m 2^{-m}$$



Proof on Kraft Inequality (2)

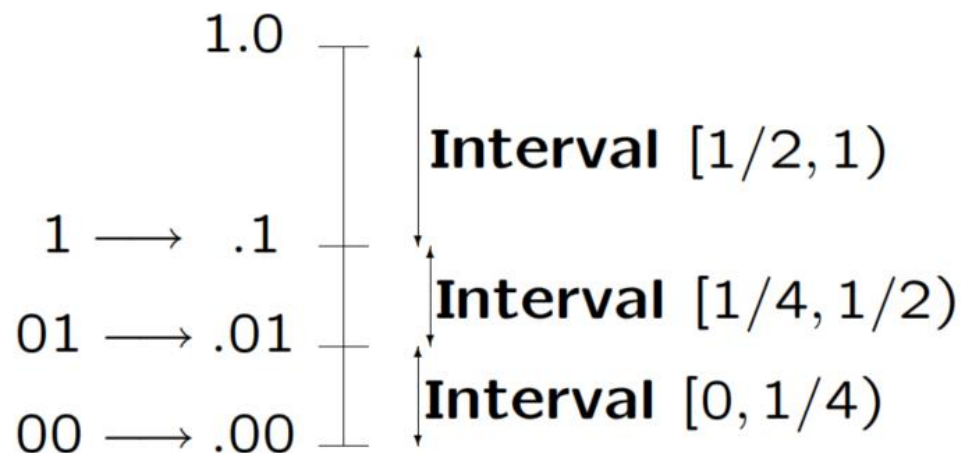
Represent binary codeword $y_1, y_2, \dots, y_m$ as

$$.y_1y_2 \cdots y_m = y_1/2 + y_2/4 + \cdots + y_m 2^{-m}$$



$\mathcal{C}(a_j)$ is a prefix of $\mathcal{C}(a_i)$ if and only if the expansion of $\mathcal{C}(a_j)$ contains the expansion of $\mathcal{C}(a_i)$ in its “approximation interval.”

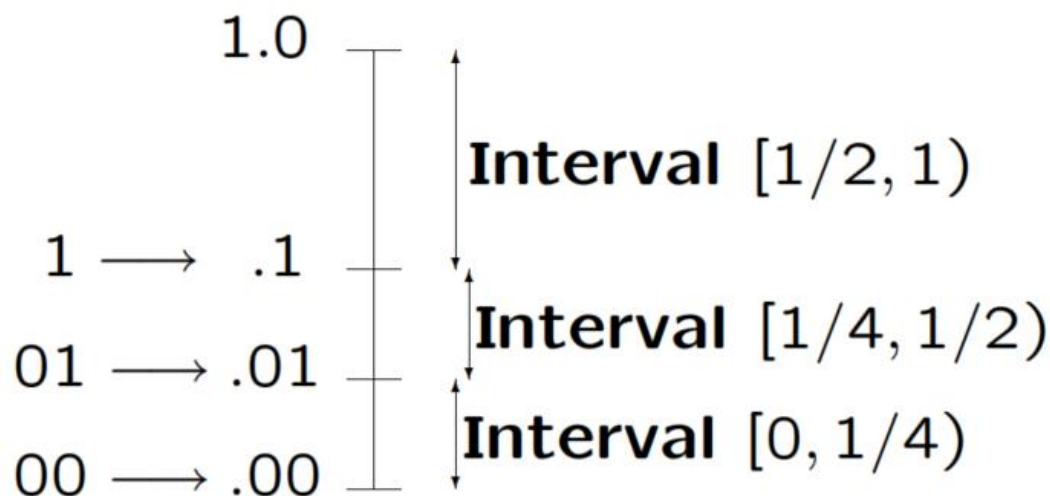
Proof on Kraft Inequality (3)



$\mathcal{C}(a_j)$ is a prefix of $\mathcal{C}(a_i)$ if and only if the expansion of $\mathcal{C}(a_j)$ contains the expansion of $\mathcal{C}(a_i)$ in its “approximation interval.”

Thus a code is a prefix code iff the base 2 approximation intervals are disjoint.

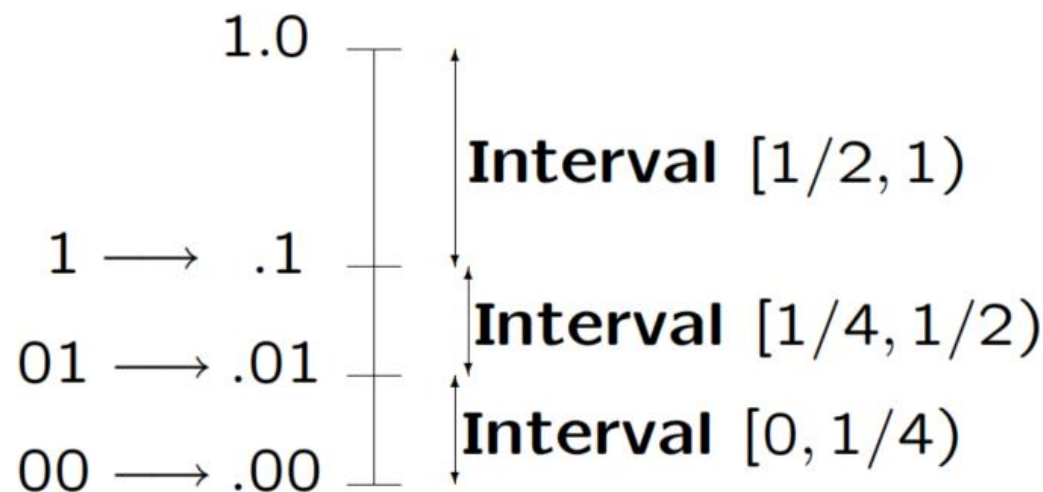
Proof on Kraft Inequality (4)



A code is a prefix code iff the base 2 approximation intervals are disjoint.

But the sum of disjoint approximation intervals is at most 1.

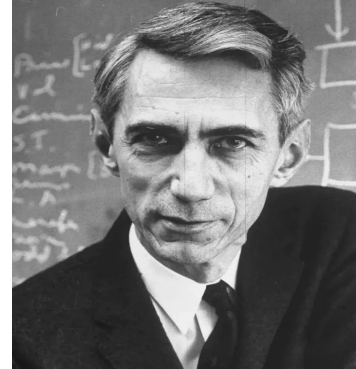
Proof on Kraft Inequality (5)



The sum of disjoint approximation intervals is at most 1.

Code is full iff approximation intervals fill up $[0, 1)$

Shannon Code



□ Binary Shannon code has the following coding steps:

(1) Sort the symbols of the info source by the descending order of their sending probabilities, e.g. $p(a_1) \geq \dots \geq p(a_n)$



(2) Let $p(a_0) = 0$ and $p_a(a_j) = \sum_{i=0}^{j-1} p(a_i)$, $j = 1, 2, \dots, n$ represents the accumulate probability before the j -th symbol.



(3) Decide the length k_i of the i -th codeword to satisfy the inequalities of $-\log_2 p(a_i) \leq k_i \leq 1 - \log_2 p(a_i)$.



(4) Represent $p_a(a_j)$ in its binary form and take k_i numbers after the decimal point as the codeword of the symbol a_i .

Example of Shannon Code

- ❑ Please encode the following single symbol info source by the binary Shannon code and calculate its coding efficiency.

$$\left(\begin{array}{c} X \\ P(X) \end{array} \right) = \left\{ \begin{array}{cccccc} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 \\ 0.25 & 0.25 & 0.2 & 0.15 & 0.1 & 0.05 \end{array} \right\}$$

Example of Shannon Code

- ❑ Please encode the following single symbol info source by the binary Shannon code and calculate its coding efficiency.

$$\begin{pmatrix} X \\ P(X) \end{pmatrix} = \begin{Bmatrix} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 \\ 0.25 & 0.25 & 0.2 & 0.15 & 0.1 & 0.05 \end{Bmatrix}$$

- Please compute expected length and coding efficiency. (89.75%)

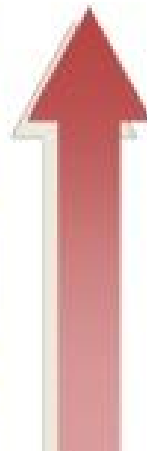
To convert a fraction, keep multiplying the fraction part by 2 until it becomes 0. Collect the integer parts in forward order.

$-\log_2 0.25 = 2$	$0.25 \times 2 = 0.5$	$0.5 \times 2 = 1.0$	$0.7 \times 2 = 1.4$	$0.85 \times 2 = 1.7$	$0.95 \times 2 = 1.9$
$(0.0)_{10} = (00)_2$	$0.5 \times 2 = 1.0$	$0.0 \times 2 = 0.0$	$0.4 \times 2 = 0.8$	$0.7 \times 2 = 1.4$	$0.9 \times 2 = 1.8$
$-\log_2 0.25 = 2$		$0.0 \times 2 = 0.0$	$0.8 \times 2 = 1.6$	$0.4 \times 2 = 0.8$	$0.8 \times 2 = 1.6$
$(0.25)_{10} = (01)_2$	$-\log_2 0.2 = 2.3219$	$-\log_2 0.15 = 2.737$		$0.8 \times 2 = 1.6$	$0.6 \times 2 = 1.2$
	$(0.5)_{10} = (100)_2$	$(0.7)_{10} = (101)_2$	$-\log_2 0.1 = 3.3219$		$0.2 \times 2 = 0.4$
			$(0.85)_{10} = (1101)_2$	$-\log_2 0.05 = 4.329$	
				$(0.95)_{10} = (11110)_2$	

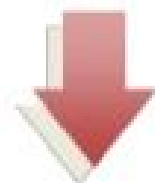
CONVERTING DECIMAL TO BINARY

- ❑ To convert a decimal integer into binary, keep dividing by 2 until the quotient is 0. Collect the remainders in *reverse* order
- ❑ To convert a fraction, keep multiplying the fractional part by 2 until it becomes 0. Collect the integer parts in *forward* order
- ❑ Example: 162.375: So, $(162.375)_{10} = (10100010.011)_2$

162 / 2	= 81	rem 0
81 / 2	= 40	rem 1
40 / 2	= 20	rem 0
20 / 2	= 10	rem 0
10 / 2	= 5	rem 0
5 / 2	= 2	rem 1
2 / 2	= 1	rem 0
1 / 2	= 0	rem 1



0.375 x 2	= 0.750
0.750 x 2	= 1.500
0.500 x 2	= 1.000



Fano Code



□ M-ary Fano code has the following coding steps:

(1) Sort the symbols of the info source by the descending order of their sending probabilities, e.g. $p(a_1) \geq \dots \geq p(a_n)$



(2) Divide the probabilities into M groups having similar sum probabilities.



(3) Each group is assigned a code-element chosen from M possible values.



(4) Consider each group as a unity and repeat steps (2) and (3) until the probabilities is undividable.

Example of Fano Code

- ❑ Please encode the following single symbol info source by the binary Fano code and calculate its coding efficiency.

$$\begin{pmatrix} X \\ P(X) \end{pmatrix} = \left\{ \begin{array}{cccccc} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 \\ 0.25 & 0.25 & 0.2 & 0.15 & 0.1 & 0.05 \end{array} \right\}$$

Coding efficiency: 98.91%

Example of Fano Code

- ❑ We have a binary discrete info source X , whose probability distribution is $P(X) = [0.7, 0.3]$, please encode its expanded source of order three by the binary Fano code and calculate its coding efficiency.
Coding efficiency:

Huffman Code



□ Binary Huffman code has the following coding steps:

(1) Sort the symbols of the info source by the descending order of their sending probabilities, e.g. $p(a_1) \geq \dots \geq p(a_n)$



(2) Assign 0 and 1 to the symbols a_{n-1} and a_n having the lowest probabilities, respectively. Combine a_{n-1} and a_n as a new symbol, whose probability is the sum probability of a_{n-1} and a_n . The new source S_1 has $(n - 1)$ symbols.



(3) Sort the symbols in the new source S_1 by the descending order of their sending probabilities. Repeat step (2) and obtain a new reduced source S_2 having $(n - 2)$ symbols.



(4) Repeat the above steps until the info source is reduced to two symbols. Get back to the original symbol along the coding path and obtain the codewords.

Examples of Huffman Coding

- ❑ Please encode the following single symbol info source by the binary Huffman code and calculate its coding efficiency.

$$\left(\begin{matrix} X \\ P(X) \end{matrix} \right) = \left\{ \begin{matrix} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 \\ 0.25 & 0.25 & 0.2 & 0.15 & 0.1 & 0.05 \end{matrix} \right\}$$

Coding efficiency: 98.91%

- ❑ A ternary discrete info source X has a probability distribution $\{p(0) = 0.5, p(1) = 0.3, p(2) = 0.2\}$, please encode its expanded source of order two by the binary Huffman code and calculate its coding efficiency.

Coding efficiency:

Two methods for Huffman Coding

- ❑ **Method 1:** The combined new symbol is put **behind** the other symbols having the same probabilities.
- ❑ **Method 2:** The combined new symbol is put **ahead of** the other symbols having the same probabilities.
- ❑ **Example:** encode the following single-symbol source by the binary Huffman code with these two methods and calculate the variance of the codeword length:

$$\left(\begin{array}{c} X \\ P(X) \end{array} \right) = \left\{ \begin{array}{ccccc} a_1 & a_2 & a_3 & a_4 & a_5 \\ 0.4 & 0.2 & 0.2 & 0.1 & 0.1 \end{array} \right\}$$

- ❑ **Conclusion:** when sorting the symbols of the reduced source, putting the combined new symbol in front is capable of efficiently using the short codewords.

English Language

Char	Prob	Char	Prob	Char	Prob	Char	Prob
␣	0.2	I	0.055	C	0.023	B	0.0105
E	0.105	R	0.054	F,U	0.0225	V	0.008
T	0.072	S	0.052	M	0.021	K	0.003
O	0.0654	H	0.047	P	0.0175	X	0.002
A	0.063	D	0.035	Y,W	0.012	J,Q	0.001
N	0.059	L	0.029	G	0.011	Z	0.001

m-ary Huffman Code

- ❑ If the codewords of an m -ary code constitute a tree graph, the number of the separable codewords is $m + k(m - 1)$, where k is positive integer.
- ❑ For minimising the average codeword length, the last reduced source should have m different symbols, when using m -ary Huffman code.
- ❑ In the first step of assigning the code-elements to the symbols having lowest probabilities, the number of symbols processed is not m .
- ❑ If a source has n symbol, let k is the minimum integer satisfying $m + k(m - 1) > n$, then $s = m + k(m - 1) - n$ codewords is abandoned.
- ❑ We then simultaneously process $(m - s)$ symbols in the first step.
- ❑ Encode $\left(\begin{matrix} X \\ p(X) \end{matrix} \right) = \left\{ \begin{matrix} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 & a_8 \\ 0.4 & 0.18 & 0.1 & 0.1 & 0.07 & 0.06 & 0.05 & 0.04 \end{matrix} \right\}$ by the ternary Huffman code and calculate its coding efficiency.

Comparison

- ❑ Shannon code, Fano code and Huffman code all consider the statistics of the info source. The symbols frequently sent are encoded by a short codeword but those infrequently sent are encoded by a long codeword in order to reduce the average codeword length.
- ❑ Shannon code has a unique coding scheme but its coding efficiency is not very high. Fano code and Huffman code both have multiple coding schemes.
- ❑ Fano code is suitable for the info source having the close group probabilities after the division. Huffman code does not have any specific requirement on the info source. It has a high coding efficiency and the complexity of its encoder is low.

One is given 7 bottles of wine. It is known that precisely one bottle has gone bad (tastes terrible). From inspection of the bottles, it is determined that the probability p_i that the i -th bottle ($i=1, \dots, 7$) is bad is given by

$$(p_1, p_2, \dots, p_7) = \left(\frac{7}{28}, \frac{6}{28}, \frac{5}{28}, \frac{4}{28}, \frac{3}{28}, \frac{2}{28}, \frac{1}{28}\right).$$

(1) Tasting will determine the bad wine. Suppose you taste the wines one at a time. Choose the order of tasting to minimize the expected number of tastings required to determine the bad bottle.

(a) What is the mean number of tastings required? [10]

(b) Which bottle should be tasted first? [3]

(2) For the first sample, you mix some of the wines in a fresh glass and sample the mixture. You proceed, mixing and tasting, stopping when the bad bottle has been determined. (Tips: Use the methodology of Huffman coding.)

(c) Please design a mixture pattern to achieve the minimum mean number of tastings required to determine the bad wine. [8]

(d) In your design, what is the mean number of tastings required to determine the bad wine? [4]